

Lab — Spam Email Classifier

Build a real spam detector using TF-IDF + Logistic Regression from scratch.

Part A — Load and explore the data

```
import pandas as pd
import numpy as np
!wget -q https://archive.ics.uci.edu/ml/machine-learning-databases/00228/smssspamcollection.zip
!unzip -o smssspamcollection.zip
!mv SMSSpamCollection SMSSpamCollection
df = pd.read_csv('SMSSpamCollection', sep='\t',
header=None, names=['label', 'message'])
print(df.head())
print('\nShape:', df.shape)
print('\nClass balance:')
print(df['label'].value_counts())
print(f'Spam rate: {747/5574*100:.1f}%')
print('\nSpam example:')
print(df[df['label']=='spam']['message'].iloc[0])
```

```
Archive: smssspamcollection.zip
  inflating: SMSSpamCollection
  inflating: readme
mv: 'SMSSpamCollection' and 'SMSSpamCollection' are the same file
label      message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...
```

Shape: (5572, 2)

```
Class balance:
label
ham      4825
spam      747
Name: count, dtype: int64
Spam rate: 13.4%
```

```
Spam example:
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)
```

Part B — Pre-process and vectorise

```
import re, nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
nltk.download(['stopwords', 'wordnet'], quiet=True)
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z ]', '', text)
    tokens = [w for w in text.split() if w not in stop_words]
    tokens = [lemmatizer.lemmatize(w) for w in tokens]
    return ' '.join(tokens)
df['clean'] = df['message'].apply(preprocess)
df['target'] = (df['label'] == 'spam').astype(int)
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2),
sublinear_tf=True)
X = tfidf.fit_transform(df['clean'])
y = df['target'].values
X_tr, X_te, y_tr, y_te = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
print(f'Train: {X_tr.shape}, Test: {X_te.shape}')
```

Train: (4457, 5000), Test: (1115, 5000)

Part C — Train and evaluate

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report,
                             confusion_matrix, accuracy_score)
import seaborn as sns
import matplotlib.pyplot as plt
clf = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
clf.fit(X_tr, y_tr)
y_pred = clf.predict(X_te)
print('Accuracy:', accuracy_score(y_te, y_pred))
print(classification_report(y_te, y_pred,
                             target_names=['Ham (genuine)', 'Spam']))
cm = confusion_matrix(y_te, y_pred)
print('Confusion matrix:')
print(cm)

```

```

Accuracy: 0.9659192825112107
      precision    recall  f1-score   support

Ham (genuine)      0.96      1.00      0.98       966
      Spam          1.00      0.74      0.85       149

   accuracy                   0.97       1115
  macro avg      0.98      0.87      0.92       1115
 weighted avg      0.97      0.97      0.96       1115

Confusion matrix:
[[966   0]
 [ 38 111]]

```

Part D — Predict new messages

```

def predict_spam(message):
    '''Predict if a new message is spam or ham.'''
    cleaned = preprocess(message)
    vector = tfidf.transform([cleaned])
    pred = clf.predict(vector)[0]
    prob = clf.predict_proba(vector)[0]
    label = 'SPAM' if pred==1 else 'HAM'
    conf = prob[pred]
    return f'{label} (confidence: {conf:.1%})'

messages = [
    'Congratulations! You won a free iPhone. Click here to claim now!',
    'Hey, are we still meeting at 6pm for dinner?',
    'URGENT: Your bank account has been suspended. Verify now!',
    'Can you pick up milk on the way home?',
]

for msg in messages:
    print(f'{predict_spam(msg):30} | {msg[:55]}...')

```

```

SPAM (confidence: 72.2%) | Congratulations! You won a free iPhone. Click here to c...
HAM (confidence: 96.9%) | Hey, are we still meeting at 6pm for dinner?...
HAM (confidence: 73.4%) | URGENT: Your bank account has been suspended. Verify no...
HAM (confidence: 97.1%) | Can you pick up milk on the way home?...

```